ML RAG AI Agent

Architecture & Interview Guide

Agentic RAG over ML PDFs + a SQL benchmark database, with evaluation and observability

1. The 30-second pitch

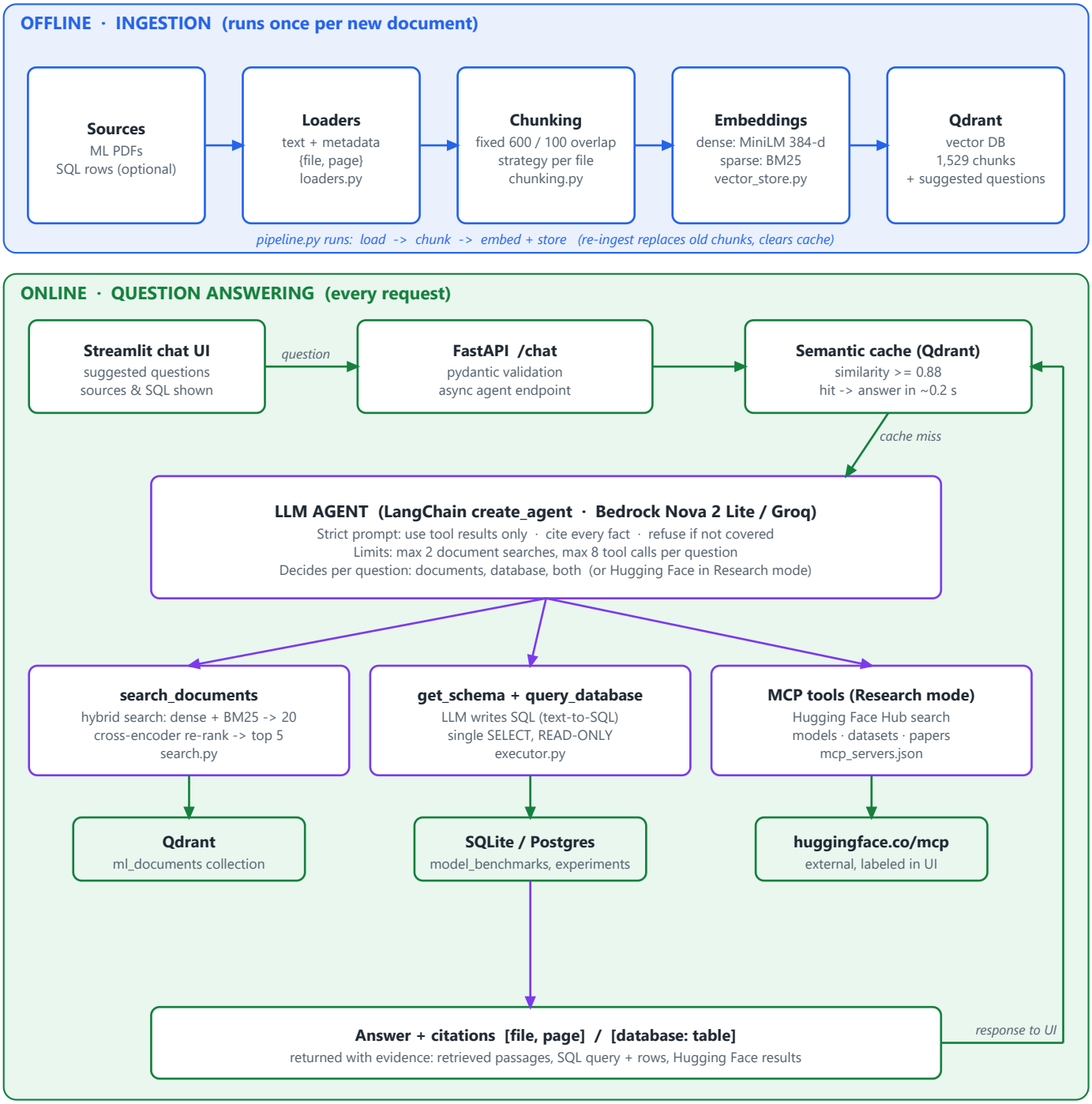
> *"I built an* **agentic RAG assistant for machine learning** *that answers* **only from a company's own data** *— a library of ML PDFs plus a SQL database of model benchmarks — with citations for every fact. Concepts come from* **hybrid retrieval with re-ranking** *over the PDFs; exact numbers come from* **text-to-SQL** *on a read-only database; an* **LLM agent** *decides which to use. It is served by* **FastAPI***, has a* **Streamlit** *chat UI, a* **semantic cache***,* **MCP** *integration for external tools, and an* **evaluation suite** *that drove every design decision — hybrid search plus re-ranking raised retrieval hit rate from 89% to 100% on my test set. The same architecture works for any domain: swap the PDFs and the tables."*

Key numbers to remember

Measure	Result
Knowledge base	387-page ML textbook → 1,529 chunks (600 chars, 100 overlap) in Qdrant
Retrieval, dense only	89% hit rate @5, MRR 0.722
Retrieval, hybrid + re-rank (chosen)	100% hit rate @5, MRR 0.847 (18 hand-labelled questions)
Chunking comparison	fixed 400 / 600 / 1000 → all 100% hit; semantic → 89%
Answer quality (LLM judge)	faithfulness 5.0 / 5, relevance 4.83 / 5, off-topic refused 2 / 2
Semantic cache	repeat question 3.7 s → 0.2 s, zero LLM cost; threshold 0.88 tuned on data
Agent latency	~3 s single-tool questions; 9–12 s multi-step (small model)

2. System architecture

Three parts: **offline ingestion** builds the knowledge base, **online question answering** runs on every request, and **quality & operations** measure and run the system.



QUALITY & OBSERVABILITY

- Golden test set: 18 questions with answer pages + 2 off-topic
- Retrieval eval: hit rate @5, MRR (dense / BM25 / hybrid / +rerank)
- Chunking eval: fixed 400 / 600 / 1000 vs semantic
- LLM-as-judge: faithfulness, relevance, refusals
- Admin inspector: per-step trace of one question
- LangSmith tracing: ready, switched on via .env

DEPLOYMENT & CONFIG

- Docker Compose: API · UI · Qdrant · Postgres
- Settings: .env -> pydantic-settings (typed)
- Secrets never in git (.gitignore)
- Admin endpoints protected by API key
- Own MCP server exposes our tools
- uv + uv.lock: reproducible installs

3. The interview script (5–7 minutes)

Follow the standard system-design order: requirements first, then data, retrieval, generation, serving, cost, quality, and finally trade-offs.

Step 1 — Requirements (say these first)

> *"**Functional:** answer ML concept questions from our PDFs, answer exact numeric questions from our benchmark database, combine both when needed, refuse anything outside our data. **Non-functional:** every answer grounded and cited; a few seconds of latency; low cost; model-agnostic so we can switch LLM providers; and measurable quality — every design choice backed by an evaluation."*

Step 2 — The key insight: two kinds of data

> *"The data comes in two shapes that need different retrieval. **Unstructured text** is best found by semantic search. **Structured numbers** must never go through vector search — embeddings can't compare numbers or compute averages. So text goes to RAG and numbers go to **text-to-SQL**, and an **agent** routes each question to the right one, or both." (I measured it: vector search over SQL rows got 'fastest model' right only by luck; SQL is exact every time.)*

Step 3 — Ingestion

> *"Loaders turn each source into documents with metadata — file and page — which become citations. For chunking I compared fixed-size, semantic and LLM-based chunking on a labelled test set: recursive 600-character chunks with 100 overlap won — 100% hit rate where semantic chunking reached 89% — at zero LLM cost. Strategy is configurable per document. Each chunk gets a dense embedding for meaning and a BM25 vector for keywords, both stored in Qdrant."*

Step 4 — Retrieval

> *"Two stages. **Hybrid search**: dense vectors catch paraphrases, BM25 catches exact jargon like StandardScaler — the small embedding model scored that sentence only 0.31 for 'how do I scale features', which is why keyword search is needed. Qdrant fuses both into 20 candidates. Then a **cross-encoder re-ranker** reads question and chunk together and keeps the top 5. Dense alone: 89% hit, MRR 0.72; hybrid: 94%; hybrid + re-rank: 100%, MRR 0.85."*

Step 5 — Generation and the agent

> *"The agent — LangChain's create_agent, a LangGraph loop — has tools for document search, database schema, read-only SQL and listing documents. Its prompt: use tool results only, cite every fact, refuse when not covered. I added hard tool-call limits after seeing the model search repeatedly — that cut some answers from 12 s to about 3 s. Answers return structured evidence — passages and the exact SQL with rows — shown in the UI so users can verify every number."*

Step 6 — Serving

> *"FastAPI with pydantic schemas. The chat endpoint is async because the agent waits on network LLM calls; CPU-bound endpoints run in FastAPI's thread pool. Models load at startup. Admin endpoints — inspection, evaluation, uploads — are separate and protected by an API key. Streamlit is a thin client, so the UI can be replaced without touching the backend."*

Step 7 — Cost and latency

> *"A semantic cache in Qdrant: a question with similarity ≥ 0.88 to an answered one returns instantly — 3.7 s to 0.2 s at zero LLM cost. I tuned 0.88 from data: rewordings scored 0.90–0.92, different questions 0.78 or lower. The cache is cleared whenever documents are re-indexed. The LLM sits behind one function, so switching Groq and Bedrock is a config change."*

Step 8 — Quality, evaluation, observability

> *"Quality is measured, not assumed: a hand-labelled golden set for retrieval metrics, a chunking benchmark, and an LLM-as-judge for faithfulness, relevance and off-topic refusals — faithfulness 5.0 / 5, off-topic refused 2 / 2. An admin inspector shows each pipeline step for a query, every response carries its tool calls and sources, and LangSmith tracing can be enabled through configuration for production monitoring."*

Step 9 — Trade-offs and next steps (always end here)

> *"Some answers take 9–12 s on a small model, so **streaming** is next; the cache is single-node, so production would use **Redis with TTL**; and I'd add **parent-child retrieval** and table-aware chunking, because my one remaining failure was a table split across chunks. To scale: Qdrant and Postgres as services, stateless API replicas behind a load balancer, a queue for ingestion, user auth and rate limiting."*

4. Tools used — and why

Layer	Tool	Why this one
Language / packaging	Python 3.12, uv	Fast, reproducible installs via uv.lock
LLM framework	LangChain (create_agent, built on LangGraph)	One interface for many LLMs, vector stores and tools; agent loop and tool-call limits built in
LLM	AWS Bedrock (Amazon Nova 2 Lite), Groq	Model-agnostic by design. Nova 2 Lite was the only tested model correct on every agent test; Groq is fastest but rate-limited on the free tier
Vector DB	Qdrant	Native hybrid search (dense + sparse in one collection), metadata filters per PDF, same API locally, in Docker and in the cloud
Embeddings	all-MiniLM-L6-v2	Free, local, fast on CPU; no data leaves the machine
Keyword search	BM25 (FastEmbed)	Catches exact terms the small embedding model misses
Re-ranker	cross-encoder ms-marco-MiniLM	Biggest precision gain per millisecond (~0.2 s for 20 chunks)
Structured data	SQLAlchemy + SQLite / Postgres	One code path for both; read-only engine for safety
API	FastAPI + pydantic	Async support, automatic validation and /docs
UI	Streamlit	A chat UI in ~100 lines of Python; easy for students to change
Tool integration	MCP (own server + Hugging Face client)	Open standard: add external tools by config, expose ours to any MCP client
Config	pydantic-settings + .env	Typed, validated settings; secrets stay out of git
Deployment	Docker Compose	API, UI, Qdrant and Postgres together, same on any machine
Quality	Custom eval + LangSmith (ready)	Our metrics answer our questions; LangSmith adds hosted tracing and experiment comparison

5. Tools not used — and why (interviewers love this)

Not used	Why not	When I would
Pinecone	Paid and closed; Qdrant has native hybrid search and runs locally for free	Fully managed scaling without running Qdrant Cloud
Chroma / FAISS	No native hybrid search in my setup; FAISS is a library (no server, metadata filters or persistence layer)	FAISS for a quick offline prototype
OpenAI embeddings	Cost, and data sent to an external API; MiniLM is free and local	If retrieval quality plateaued — the eval would measure the gain
Fine-tuning the LLM	RAG is cheaper, updates instantly and gives citations; fine-tuning teaches style, not reliable facts	For tone or format, never for knowledge
Semantic / agentic chunking as default	Measured worse (89%) or far more expensive (an LLM call per page)	Per document, if its eval says so
LlamaIndex	LangChain covered every need and matched my experiments	If I needed its advanced indexing
Explicit LangGraph graph	create_agent is LangGraph underneath; no custom branching needed yet	Multi-step workflows with branching or human approval
Bedrock Knowledge Bases / AgentCore	They'd hide chunking, retrieval and evaluation — the parts I wanted to control and measure; AgentCore hosts agents, which FastAPI already does	Managed production on AWS with less custom code
Claude on Bedrock	Account's Marketplace subscription failed (known India/UPI payment issue); code is model-agnostic, so I used Nova	Once the account issue is fixed — a config change
unstructured (hi_res / OCR)	Heavy dependencies; my PDFs are text-based	Scanned PDFs
Redis	Single-node prototype; the Qdrant cache was enough	Production: shared cache, TTL, eviction
Code-execution sandbox	Running LLM-written code in the API process is a security risk	In an isolated container via MCP
Kubernetes	Overkill at this scale	Many replicas, autoscaling
React frontend	Streamlit is faster to build and to teach	A customer-facing product

6. LLM hallucination

What it is: an LLM stating something **fluent and confident but false or unsupported**. An LLM predicts plausible next words and has no built-in "I don't know". Common types:

- **Made-up facts** — "Random Forest scored 98.2%" when no such number exists.
- **Unfaithful to the context** — the right passage was retrieved, but the answer adds or changes details.
- **Fake citations** — "page 212" when the fact isn't on page 212.
- **Answering outside scope** — using general internet knowledge when the app should use only your data.

How this project reduces it

Layer	What it does	Where
1. Grounding (RAG)	The LLM answers from retrieved passages, not from memory	search.py, rag.py
2. Better retrieval	Missing passages tempt the LLM to fill gaps; hybrid + re-rank found the right page in 100% of tests	search.py
3. Exact numbers via SQL	Numbers come from the database, never from LLM memory or vector search	tools.py, executor.py
4. Strict prompt rules	Tool results only, no outside knowledge, say when not covered	prompts.py
5. Mandatory citations	Every fact needs [file, page] or [database: table], so it can be checked	prompts.py
6. Refusal path	Off-topic questions get "not covered" instead of a guess (2 / 2 in tests)	prompts.py, rag.py
7. Evidence to the user	UI shows the passages and SQL rows behind each answer	agent.py, ui/app.py
8. Deterministic settings	temperature = 0 on Groq (Nova on Bedrock uses its default)	llm.py
9. Measurement	LLM judge scores faithfulness — 5.0 / 5	llm_eval.py
10. Labelled outside data	Research mode marks Hugging Face facts separately	prompts.py, ui/app.py

> **Say it honestly:** *these layers* **reduce** *hallucination a lot, but no system* **eliminates** *it. That is why we measure it (evaluation), watch it (observability), and give users citations to check.*

7. Observability, traceability and LangSmith

Term	Meaning
Observability	Understanding what the system is doing and why, from its outputs — logs, metrics and traces: latency per step, token cost, error rate, cache hit rate, answer quality over time
Traceability	Following one specific request end to end — which chunks were retrieved, which tools ran with which inputs, what the LLM saw and answered — to explain why an answer came out that way
Trace	That story as a tree of steps, each with inputs, outputs and timing
LangSmith	LangChain's hosted platform for tracing (clickable step trees with tokens and timings), evaluation (datasets, experiments compared side by side) and monitoring (latency, cost, feedback over time)

What is implemented in this project

Capability	Status	How
Per-request trace on demand	Done (ours)	run_server.py inspect "question" / GET /admin/inspect — 20 hybrid candidates → re-ranked order and scores → exact prompt → answer
Evidence in every response	Done (ours)	/chat returns tools_used, sources (passages + scores) and sql (queries + rows)
Quality metrics	Done (ours)	eval (hit rate, MRR), eval --chunking, eval --llm; saved in data/eval_results/
Index health	Done (ours)	stats / /admin/stats, chunks --page
Hosted tracing of live traffic	Ready, not on	LangSmith: LANGSMITH_TRACING=true + API key in .env (no code change for the agent)
Our own steps in hosted traces	Planned	@traceable on hybrid search, re-rank, cache lookup, RAG answer, agent entry point
Experiment comparison UI	Planned	Upload golden set as a LangSmith dataset + evaluate()

Does observability stop hallucination?

No — it finds hallucinations; the design prevents them.

- **Detect** — the trace shows exactly what the LLM saw; a claim not in the retrieved chunks was invented.
- **Diagnose** — retrieval failure (right chunk not found → fix chunking / search) or generation failure (chunk present but ignored → fix prompt / model). The LDA "topic 8" failure was diagnosed this way: the right page was retrieved but its table was split across chunks — a chunking problem.
- **Prevent regressions** — run the faithfulness eval before every model or prompt change; don't ship if it drops.
- **Monitor** — online evaluators can score live answers for groundedness and flag bad ones for review.

> *"Grounding, strict prompts, exact SQL and mandatory citations* **reduce** *hallucination; evaluation* **measures** *it before release; tracing* **detects and explains** *it in production. Observability closes the loop — it tells me which layer to fix."*

8. Likely follow-up questions

Question	Answer
Why not paste the whole book into a big context window?	Cost and latency per question, worse accuracy when the relevant part is buried in noise, and no per-fact citations. Retrieval sends ~5 relevant chunks.
How do you know your chunk size is right?	Measured: 400 / 600 / 1000 all 100% hit, MRR within 0.02 — a tie, so I kept the middle. Semantic chunking was worse (89%).
What happens with a brand-new PDF?	Drop it in, write ~15 test questions, run the chunking comparison, record the winner in config/chunking.json, ingest, re-run the evals.
How do you stop SQL injection or data damage?	Only a single SELECT is accepted, and it runs on a read-only database connection; anything else is rejected before and at the database.
What's your biggest weakness?	Multi-step latency (9–12 s on a small model) → streaming and prompt caching; single-node cache → Redis; tables split across chunks → table-aware / parent-child retrieval.
How would you scale to 10,000 users?	Qdrant cluster + managed Postgres, stateless FastAPI replicas behind a load balancer, Redis cache, a queue for ingestion, streaming, auth + rate limiting, LangSmith / Langfuse monitoring.
Why an agent instead of a fixed pipeline?	Questions mix concepts and numbers; the agent picks tools per question. Less predictable — controlled with tool limits and strict prompts. Documents-only questions can use the fixed /ask pipeline.
How does MCP fit in?	A standard way to connect tools: we add external tools (Hugging Face) by config and expose our own search / SQL tools so other AI clients can use our knowledge base.